BCSE410L – Cyber Security Project

Distributed IoT Honeypot on ESP32 with Kill-Chain Attack Classification and Real-Time Analytics

23BCE0578 Sujatro Ganguli

23BCE0603 Ishan Tare

23BCE0606 Sukruta Nadkarni

Under the Supervision of

Dr Anto S

School of Computer Science and Engineering (SCOPE)

B.Tech.

in

Computer Science and Engineering

School of Computer Science and Engineering



October 2026

ABSTRACT

This paper presents a novel IoT honeypot system implemented on the ESP32 microcontroller platform, combining multi-protocol emulation across five services (Telnet, SSH, HTTP, TLS, MQTT) with a 47-pattern kill-chain attack classification engine distributed between on-device heuristic inference and a backend probabilistic classifier. Stateful, multi-turn protocol interactions are sustained within the 520 KB SRAM constraint through static allocation and interrupt-driven I/O via AsyncTCP, featuring an interactive SSH emulation mechanism. Events are serialised in both JSON and Protocol Buffer binary formats, achieving a 67% bandwidth reduction, and are delivered reliably under adverse Wi-Fi conditions via exponential backoff. A containerized analytics backend—comprising Mosquitto, Telegraf, InfluxDB 2.7 and Grafana 10.4—receives pre-classified, selfcontained events and auto-provisions two dashboards covering 19 visualisation panels including kill-chain state timelines and MITRE ATT&CK coverage charts. Validation uses a five-step pipeline health-check script and a synthetic attack simulator covering all 47 pattern classes. Per-node hardware cost is strictly sub-Rs.450, enabling economically viable large-scale IoT threat monitoring at the network edge.

Keywords - IoT security, honeypot, ESP32, attack classification, Mirai, Hawkes process, Grafana, MQTT, embedded systems, threat intelligence.

TABLE OF CONTENTS

Sl.No	Contents	Page No.
	Abstract	**i**
1.	**INTRODUCTION**	**1**
	1.1 Background	**1**
	1.2 Motivations	**1**
	1.3 Scope of the Project	**1**
2.	**PROJECT DESCRIPTION AND GOALS**	
	2.1 Literature Review	
	2.2 Research Gap	
	2.3 Problem Statement	
	2.4 Objectives	
	2.5 Project Plan	
3.	**TECHNICAL SPECIFICATION**	
	3.1 Requirements	
	3.1.1 Functional	
	3.1.2 Non-Functional	
	3.2 Feasibility Study	
	3.2.1 Technical Feasibility	
	3.2.2 Economic Feasibility	
	3.2.2 Social Feasibility	
	3.3 System Specification	
	3.3.1 Hardware Specification	
	3.3.2 Software Specification	
4.	**DESIGN APPROACH AND DETAILS**	
	4.1 System Architecture (Not Flow diagram/flow chart)	
	4.2 Design	
	4.2.1 Algorithm1	
	4.2.2 Algorithm 2	
5.	**EXPERMENTAL SETUP AND OUTPUT**	
	5.1 Experimental Setup	

	5.2 Output of the Proposed System	
6.	**RESULT ANALYSIS**	
	6.1 Comparison with existing Systems	
7.	**CONCLUSION AND FUTURE WORKS**	
8.	**REFERENCES**	

1. INTRODUCTION

1.1 Background

The rapid growth of the Internet of Things (IoT) has created a large and continuously expanding attack surface. Many IoT devices operate with unpatched firmware, default credentials, and exposed network services, making them attractive targets for automated attacks and botnets such as Mirai, Hajime, Mozi, and Gafgyt. Honeypots provide an effective way to study these threats by presenting simulated vulnerable services and recording real attacker behaviour.

However, conventional high-interaction honeypots often require significant computing resources, while lightweight embedded honeypots may support only limited protocols and interactions. The proposed project addresses these limitations through a distributed IoT honeypot implemented on the ESP32 microcontroller. The system emulates five services—Telnet, SSH, HTTP, TLS, and MQTT—and supports stateful interactions with attackers. It also incorporates a 47-pattern attack classification system based on IoT botnet behaviour. Events are processed on the ESP32 and through a backend analytics platform using MQTT, InfluxDB, and Grafana, enabling real-time monitoring and visualization of attacks.

1.2 Motivation

The motivation for this project comes from the increasing frequency and sophistication of attacks against IoT devices. The Mirai botnet demonstrated how easily devices using default credentials could be compromised and combined into large-scale botnets capable of launching powerful attacks. Subsequent botnets have adopted more advanced scanning, exploitation, and command-and-control techniques, making continuous observation of IoT attack behaviour increasingly important.

Existing honeypot solutions face a trade-off between interaction fidelity, analytical capability, and deployment cost. High-interaction systems provide detailed attacker information but require considerable infrastructure, while lightweight embedded solutions often provide limited interaction and depend heavily on external systems for analysis. This project is motivated by the need for an affordable, scalable solution that can operate directly at the network edge.

By combining multi-protocol emulation, on-device attack classification, reliable event transmission, and real-time dashboards, the system aims to capture meaningful attack behaviour while remaining within the ESP32's resource constraints. Its low per-node hardware cost also makes large-scale deployment more practical.

1.3 Scope of the Project

The project focuses on designing and implementing a distributed IoT honeypot using the ESP32 platform for capturing, classifying, and visualizing malicious network activity. The

honeypot emulates five commonly targeted services: Telnet on port 23, SSH on port 22, HTTP on port 80, TLS on port 443, and MQTT on port 1883. It supports stateful interactions, including credential attempts, authenticated shell activity, exploitation attempts, and MQTT interactions.

The project also covers a 47-pattern attack classification framework organized into credential brute force, reconnaissance and scanning, vulnerability exploitation, post-exploitation activity, and timing/protocol anomalies. Classification is divided between deterministic on-device heuristics and backend analysis using HMM, Hawkes-process-based timing analysis, and DBSCAN anomaly detection.

For analytics, the scope includes MQTT-based event transport, Telegraf, InfluxDB, a FastAPI classifier service, and Grafana dashboards for real-time monitoring and historical analysis. System validation includes pipeline health checks and a synthetic attack simulator covering all 47 attack patterns. The project does not focus on unrestricted high-interaction SSH sessions, dynamic honeypot persona rotation, or unrestricted Internet deployment.

2. PROJECT DESCRIPTION AND GOALS

2.1 Literature Review

1. **Pa et al. – "IoTPOT: A Novel Honeypot for Revealing Current IoT Threats" (2016):** IoTPOT was developed to observe Telnet-based attacks against IoT devices. It supported multiple CPU architectures and demonstrated the presence of several IoT malware and DDoS families. This work establishes the importance of realistic Telnet emulation for IoT honeypots.
2. **Zhang et al. – "SIPHON: Towards Scalable High-Interaction Physical Honeypots" (2017):** SIPHON investigated scalable physical honeypots and showed how a physical IoT device could be used to represent multiple network endpoints. Its approach provides useful insights into scalable honeypot deployment, although the infrastructure requirements can be significant.
3. **Kolias et al. – "DDoS in the IoT: Mirai and Other Botnets" (2017):** This study examined the Mirai botnet and its impact on IoT security. It demonstrated how weak credentials and exposed services can allow large numbers of IoT devices to be recruited into DDoS botnets. The work provides an important foundation for credential-based attack detection.
4. **Herwig et al. – "Measurement and Analysis of Hajime, a Peer-to-Peer IoT Botnet" (2019):** This research analyzed Hajime's propagation and command-and-control behaviour. It highlighted that IoT botnets can use different scanning rates and propagation mechanisms, motivating behavioural rather than purely signature-based classification.
5. **Williams and Patterson – "Comparative Analysis of Low-Interaction and High-Interaction Honeypot Architectures for IoT Security Monitoring" (2020):** The study compared different levels of honeypot interaction and highlighted the trade-off between realism, resource requirements, and deployment cost. This supports the use of a medium-interaction architecture for resource-constrained IoT environments.
6. **Lee, Abdullah and Jhanjhi – "A Review on Honeypot-based Botnet Detection Models for Smart Factory" (2020):** This work reviewed the use of honeypots for detecting botnets in smart-factory environments. It emphasizes the potential of honeypots for identifying malicious activity in IoT-based industrial environments.
7. **Antonakakis et al. – "Understanding the Mirai Botnet" / Mirai-related measurement studies:** Research into Mirai has shown that large-scale IoT attacks are strongly associated with automated scanning, default credentials, and repeated compromise attempts. These findings support the use of credential and behavioural patterns in IoT honeypot classification.
8. **Franco et al. – "A Survey of Honeypots and Honeynets for Internet of Things, Industrial Internet of Things, and Cyber-Physical Systems" (2021):** This comprehensive survey classified existing IoT honeypot approaches and identified important design factors, including interaction level, deployment architecture, scalability, and data collection. It also highlighted several open challenges in IoT honeypot research.
9. **Zeng et al. – "Using Honeypots to Model Botnet Attacks on the Internet of Medical Things" (2022):** This research analyzed large-scale IoT attack sessions and identified temporal patterns in botnet behaviour. The study is particularly relevant to the proposed project because its behavioural clustering forms the

theoretical basis for the 47-pattern taxonomy used in the proposed classification system.

10. **"Modeling and Analyzing Attacker Behavior in IoT Botnet Using Temporal Convolution Network" (2022):** This work used multiple honeypots and heterogeneous attack events, including network traffic, SSH logins, file downloads, and attacker commands, to model attacker behaviour. It demonstrates the value of combining multiple event types instead of relying only on network-level features.
11. **Negera et al. – "Review of Botnet Attack Detection in SDN-Enabled IoT Using Machine Learning" (2022):** This review examined machine-learning approaches for IoT botnet detection and highlighted the importance of intelligent analysis for evolving botnet behaviour.
12. **"A Comprehensive Survey on IoT Attacks: Taxonomy, Detection Mechanisms and Challenges" (2024):** This survey discusses IoT attack categories and detection mechanisms, including honeypots, and distinguishes low-, medium-, and high-interaction honeypots. It reinforces the need for approaches that balance attack visibility with implementation complexity.

2.2 Research Gap

The reviewed studies demonstrate that IoT honeypots are effective for collecting attack intelligence, but several limitations remain. Existing solutions frequently concentrate on a single protocol, require relatively powerful infrastructure, or perform most of their analysis outside the IoT device. Furthermore, many systems focus on attack detection without providing detailed kill-chain classification and real-time visualization.

The proposed project addresses these gaps by combining **low-cost ESP32 hardware, multi-protocol stateful emulation, on-device classification, backend behavioural analysis, and real-time Grafana visualization**. The system therefore aims to provide a practical and scalable IoT threat-monitoring platform suitable for deployment at the network edge.

2.3 Problem Statement

IoT devices are increasingly exposed to cyberattacks because of weak credentials, vulnerable services, unpatched firmware, and publicly accessible network interfaces. Existing IoT honeypot solutions often have limitations such as supporting only a single protocol, providing limited attacker interaction, requiring relatively powerful hardware, or depending entirely on external infrastructure for attack classification. These limitations make them less suitable for low-cost, network-edge deployment.

Therefore, there is a need for a **low-cost, resource-efficient IoT honeypot** capable of simulating multiple commonly targeted services, capturing stateful attacker interactions, classifying attacks according to their kill-chain behavior, and providing real-time security analytics. The project addresses this problem by developing an ESP32-based distributed honeypot that combines multi-protocol emulation, on-device and backend attack classification, reliable event transmission, and real-time visualization.

2.4 Objectives

(Times New Roman 13, Bold, Title Case, Line spacing 1.5)

1. To develop a low-cost IoT honeypot using the ESP32 platform, targeting a per-node hardware cost below Rs.450.
2. To implement simultaneous emulation of five IoT services: Telnet, SSH, HTTP, TLS, and MQTT.
3. To provide stateful, multi-turn attacker interaction, including authentication, shell commands, and exploitation attempts.
4. To implement a 47-pattern attack classification system covering credential attacks, reconnaissance, exploitation, post-exploitation, and timing/protocol anomalies.
5. To perform real-time classification of suitable attack patterns directly on the ESP32 while using backend analysis for multi-session and temporal attack patterns.
6. To establish reliable transmission of security events using MQTT with JSON and Protocol Buffer serialization.
7. To develop a backend analytics pipeline using Mosquitto, Telegraf, InfluxDB, FastAPI, and Grafana for real-time monitoring and visualization.
8. To validate the complete system using automated pipeline health checks and a synthetic attack simulator covering all 47 attack classes.

2.5 Project Plan

(Times New Roman 13, Bold, Title Case, Line spacing 1.5)

<Contents, Times New Roman 12, Line spacing 1.15>

<A Gantt chart helps in organizing tasks, setting deadlines, and tracking progress, making it easier to manage the project and communicate the plan to stakeholders.>

/ Sample **/**

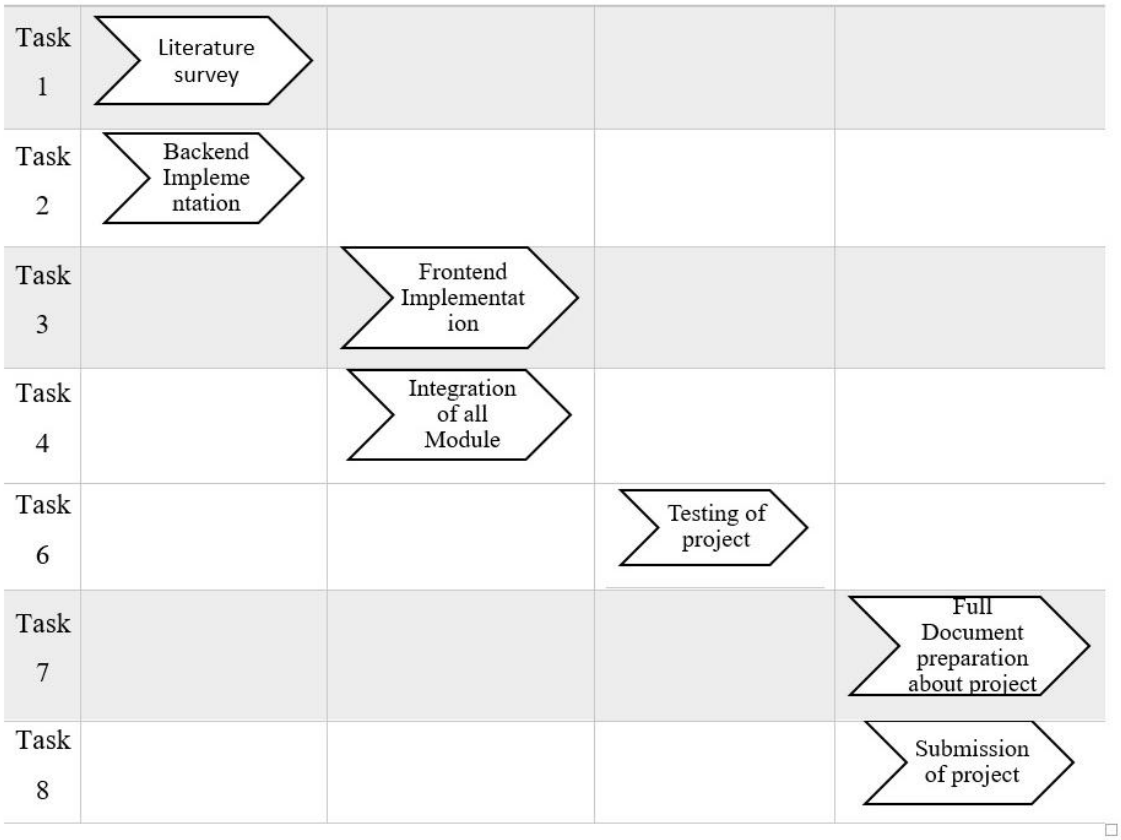


Fig. 1. Gantt chart

3. TECHNICAL SPECIFICATION

3.1 Requirements

3.1.1 Functional

- **Data Collection: The system should be able to collect attack and interaction data from Telnet, SSH, HTTP, TLS, and MQTT services.**
- **Connection Monitoring: The system should detect and record incoming connections to each emulated service.**
- **Credential Capture: The system should record attempted usernames and passwords used during authentication attempts.**
- **Interactive Session Handling: The system should support stateful, multi-turn interactions with attackers rather than terminating the session after a single request.**

- **Command Capture**: The system should record commands entered by attackers during interactive sessions.
- **HTTP Request Monitoring**: The system should capture requests to simulated endpoints such as /login, /admin, and /cgi-bin/.
- **MQTT Activity Monitoring**: The system should parse MQTT CONNECT and PUBLISH messages and record relevant interaction information.
- **Event Generation**: The system should generate structured honeypot events whenever significant attacker activity occurs.
- **Event Queuing**: The system should temporarily store events in a circular queue when immediate transmission is not possible.
- **Event Transmission**: The system should transmit collected events from the ESP32 to the backend using MQTT.
- **Reliable Delivery**: The system should retry failed event transmissions using an exponential backoff mechanism.
- **Data Compression**: The system should support Protobuf encoding to reduce network bandwidth compared with JSON.
- **Attack Classification**: The system should classify observed activity into the defined 47 attack patterns.
- **On-Device Classification**: The ESP32 should perform lightweight classification for supported credential, reconnaissance, and post-exploitation patterns.
- **Backend Classification**: The backend should perform computationally heavier classification for vulnerability exploitation and timing/protocol anomaly patterns.
- **Attack-State Analysis**: The backend should analyze attacker behavior using state-based classification techniques.
- **Anomaly Detection**: The system should identify unusual or previously unseen attack behavior using anomaly-detection mechanisms.
- **Real-Time Dashboard**: The system should display collected events and attack classifications through Grafana dashboards.

- **System Health Monitoring**: The system should provide mechanisms for checking the health and connectivity of the honeypot and backend components.
- **Attack Simulation**: The system should support synthetic attack simulations for validating the implemented attack patterns.
- **Validation**: The system should verify that all 47 defined attack patterns can be detected and classified.

3.1.2 Non-Functional

- **Performance**: The system should process and classify lightweight attack events with minimal latency while maintaining stable ESP32 operation.
- **Real-Time Operation**: Attack events should be transmitted and visualized with minimal delay to support real-time monitoring.
- **Resource Efficiency**: The honeypot should operate within the ESP32's limited **520 KB SRAM** and available processing resources.
- **Memory Stability**: The system should maintain stable memory usage during continuous operation and minimize heap fragmentation.
- **Reliability**: The system should continue collecting events even when temporary network or backend failures occur.
- **Fault Tolerance**: Failed event transmissions should be retried automatically using exponential backoff.
- **Scalability**: The architecture should support deployment of multiple ESP32 honeypot nodes connected to a centralized analytics backend.
- **Low Cost**: The system should maintain a low per-node deployment cost of approximately **₹450 or less**.
- **Bandwidth Efficiency**: The system should minimize network bandwidth consumption through compact event representation such as Protocol Buffers.
- **Availability**: The honeypot services and analytics components should remain operational during continuous traffic attacks.
- **Maintainability**: The software should be organized into separate protocols, classification, communication, and analytics components to simplify maintenance.
- **Extensibility**: The architecture should allow additional attack patterns, protocols, or analytics capabilities to be incorporated in future versions.
- **Security**: The honeypot should isolate and monitor attacker interactions without exposing the underlying management infrastructure.
- **Usability**: The Grafana dashboards should present attack activity and system status in an understandable visual format.
- **Data Integrity**: Collected attack events should preserve important information such as timestamps, source information, protocol, authentication activity, commands, and classification results.
- **Continuous Operation**: The system should be capable of running continuously without significant degradation in performance or memory stability.
- **Testability**: The system should provide mechanisms for health checks and synthetic attack testing to verify individual components and the complete system.

3.2 Feasibility Study

3.2.1 Technical Feasibility

- **Technology Availability**: The project uses readily available technologies such as the ESP32 Dev Kit v1, Wi-Fi, MQTT, Docker, Mosquitto, Telegraf, InfluxDB, FastAPI, and Grafana, making the system technically feasible to develop and deploy.
- **Hardware Feasibility:** The ESP32 provides a 240 MHz dual-core processor, 520 KB SRAM, and 4 MB flash, which is sufficient for implementing lightweight honeypot services and on-device attack classification.
- **Protocol Support:** The system can implement five commonly targeted protocols—Telnet, SSH, HTTP, TLS, and MQTT—to provide multi-protocol honeypot functionality.
- **Software Feasibility:** The required software components are based on established networking, database, containerization, and visualization technologies, making implementation practical.
- **Performance Feasibility:** The prototype maintains approximately 180–220 KB of free heap during steady-state operation, while on-device classification takes less than 10 µs, demonstrating that the ESP32 can support the proposed workload.

3.2.2 Economic Feasibility

- **Low Hardware Cost:** The project uses an ESP32 Dev Kit v1 costing approximately ₹400, with a Micro-USB connection costing approximately ₹50, keeping the hardware cost around ₹450 per honeypot node.
- **Cost-Benefit Analysis:** The low-cost hardware makes it possible to deploy multiple honeypot nodes without requiring expensive specialized cybersecurity hardware.
- **Open-Source Technologies:** The backend uses technologies such as Mosquitto, Telegraf, InfluxDB, Grafana, FastAPI, and Docker, reducing software licensing expenses.
- **Scalable Deployment:** The low per-node cost allows additional ESP32 honeypots to be deployed as monitoring requirements increase, providing a cost-effective distributed security monitoring approach.
- **Infrastructure Efficiency:** Lightweight processing on the ESP32 reduces the need for powerful hardware at every honeypot node, while computationally intensive analysis can be performed by the centralized backend.

3.2.3 Social Feasibility

- **User Acceptance:** The system provides security analysts with real-time attack information through Grafana dashboards, making attack activity easier to monitor and understand.

- **Ease of Monitoring:** Visual dashboards allow users to observe honeypot events, attack patterns, and system activity without manually examining raw network logs.
- **Cybersecurity Awareness:** The project can help security teams understand common IoT attack behaviors, including credential attacks, reconnaissance, exploitation, and post-exploitation activity.
- **Educational Value:** The low-cost ESP32 implementation makes the system suitable for academic cybersecurity research, experimentation, and demonstration.
- **Operational Safety:** The honeypot is designed to attract and observe malicious interactions rather than expose real IoT services, allowing security personnel to study attacker behavior in a controlled environment.
- **Future Adoption:** Its low cost, multi-protocol support, and real-time analytics make the approach suitable for organizations or educational institutions seeking an accessible IoT security monitoring solution.

3.2 System Specification

3.2.1 Hardware Specification

- Microcontroller: The system should use an ESP32 Dev Kit v1 with a 240 MHz dual-core Xtensa LX6 processor, 520 KB SRAM, and 4 MB flash memory.
- Power Supply: The ESP32 should be powered through a Micro-USB connection.
- Memory Management: The system should use static memory allocation where possible to reduce heap fragmentation and maintain stable operation.
- Network Connectivity: The ESP32 should provide Wi-Fi connectivity for communication with the analytics backend.
- Cost: The per-node hardware cost should remain below approximately ₹450.

3.2.2 Software Specification

(Times New Roman 12, Bold with Italic, Title Case, Line spacing 1.5)

<Contents, Times New Roman 12, Line spacing 1.15>

- **Operating System:** FreeRTOS / Linux or Windows
- **Programming Languages:** C/C++, Python
- **Development Environment:** Arduino IDE, Docker
- **Libraries and Frameworks:** AsyncTCP, FastAPI
- **Database:** InfluxDB
- **Communication:** MQTT
- **Visualization:** Grafana
- **Security Tools:** Honeypot, Attack Classifier

4. DESIGN APPROACH AND DETAILS

4.1 System Architecture

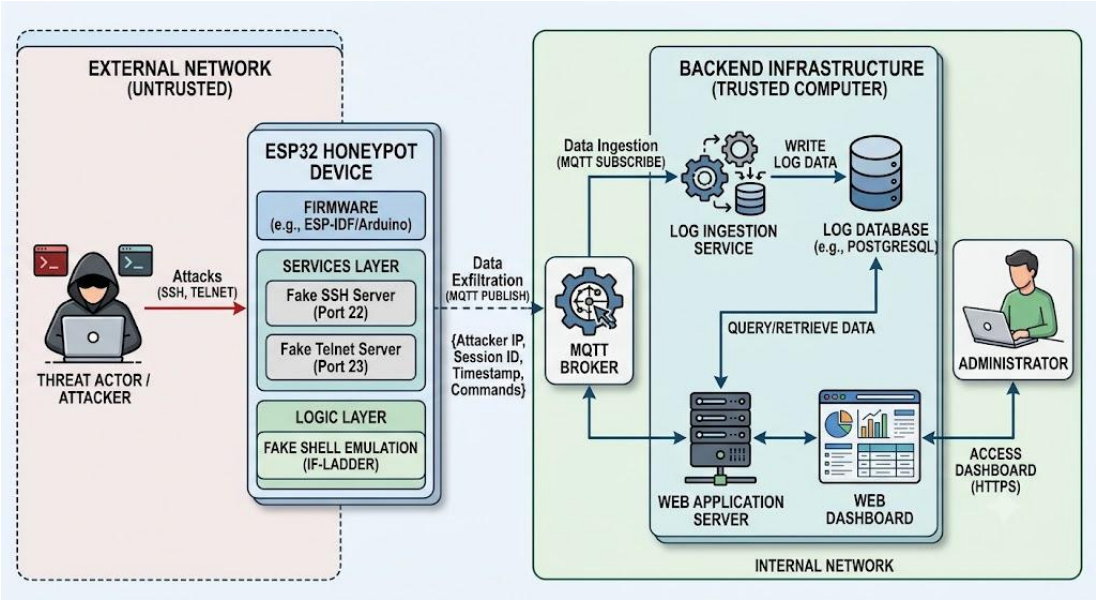


Fig. 2. System Architecture

The system architecture consists of an **ESP32-based honeypot deployed in an external untrusted network** and a **trusted backend infrastructure**. Attackers interact with the ESP32 through fake SSH and Telnet services, while the shell-emulation layer records their activities. The captured information, including the attacker's IP address, session ID, timestamp, and commands, is transmitted through an **MQTT broker** to the backend. The log ingestion service processes the received data and stores it in the database. A web application retrieves the stored information and presents it through a **web dashboard**, which allows the administrator to monitor and analyze attacker activities securely.

4.2 Design

4.2.1 Algorithm 1

Kill-Chain Attack Classification

Input: Network activity, credentials, commands, requests, and protocol events

Output: Classified attack pattern

1. Start the honeypot and monitor incoming network connections.
2. Capture attacker activity from Telnet, SSH, HTTP, TLS, and MQTT services.
3. Generate a structured event for each significant activity.
4. Check the event against the predefined 47 attack patterns.
5. Perform lightweight classification on the ESP32 for supported patterns.
6. Send complex events to the backend through MQTT.
7. Use HMM and Viterbi to identify attacker behavior sequences.
8. Use Hawkes Process to analyze timing-based attack behavior.
9. Apply DBSCAN and Z-score analysis to identify anomalous or previously unseen behavior.
10. Store the classification results in InfluxDB.
11. Display attack activity and classification results on Grafana.
12. End or continue monitoring for the next event.

This follows the project's division of the 47 attack patterns between on-device and backend classification and its use of HMM, Viterbi, Hawkes-process analysis, and DBSCAN/Z-score anomaly detection.

4.2.2 Algorithm 2

Reliable Event Transmission

Input: Honeypot event
Output: Successfully transmitted event

1. Generate a honeypot event.
2. Store the event in the circular event queue.
3. Encode the event using JSON or Protobuf.
4. Publish the event through MQTT.
5. Check whether transmission is successful.
6. If successful, remove the event from the queue.
7. If unsuccessful, wait using **exponential backoff**.
8. Retry transmission with increasing delay up to the defined limit.
9. Continue monitoring for new events.

This algorithm is based on the project's event queue, JSON/Protobuf encoding, MQTT delivery, and **1–30 second exponential backoff** mechanism.

5.EXPERMENTAL SETUP AND OUTPUT

5.1 Experimental Setup

The experiment was conducted using an **ESP32 Dev Kit v1** connected to a Wi-Fi network. The ESP32 was configured as a multi-protocol honeypot supporting **Telnet, SSH, HTTP, TLS, and MQTT** services. Attacker interactions were captured as structured events and transmitted to the backend using MQTT. The backend consisted of **Mosquitto, Telegraf, InfluxDB, FastAPI, and Grafana**, deployed using Docker Compose. A synthetic attack simulator was used to generate activities covering all **47 attack patterns**, while Grafana dashboards were used to monitor and analyze the collected results.

5.2 Output of the Proposed System

The proposed system successfully demonstrated multi-protocol IoT honeypot operation, attack classification, reliable event transmission, and real-time analytics. The ESP32 maintained stable operation with 180–220 KB free heap, while the on-device classification required less than 10 µs. The system validated all 47 attack patterns using a synthetic attack simulator.

Parameter	**Output**
Supported Protocols	5
Attack Patterns	47
Free Heap	180–220 KB
Event Queue	50 entries
JSON Event Size	≈180 bytes
Protobuf Event Size	≈60 bytes
Bandwidth Reduction	67%
Classification Time	<10 µs
Data Retention	365 days
Visualization Panels	19
Hardware Cost/Node	< ₹450

6. RESULT ANALYSIS

Parameter	IoTPOT	SIPHON	Conventional Honeypot	Proposed System
Protocols Supported	1	Multiple	1	5
Stateful Interaction	Limited	Yes	Limited	Yes
Attack Patterns	—	—	—	47
On-Device Classification	No	No	No	Yes
Real-Time Analytics	Limited	Limited	Limited	Yes
Attack Visualization	No	Limited	No	19 panels
Hardware Cost/Node	High	High	Varies	< ₹450
Event Encoding	—	—	—	JSON + Protobuf
Bandwidth Reduction	—	—	—	67%

7. CONCLUSION AND FUTURE WORKS

The proposed system successfully implements a **low-cost distributed IoT honeypot using ESP32** with support for **Telnet, SSH, HTTP, TLS, and MQTT**. It provides stateful multi-turn interaction and classifies attacker activity into **47 kill-chain attack patterns**. The system combines on-device classification with backend analysis and provides reliable event transmission using MQTT, JSON, and Protobuf. The backend integrates Mosquitto, Telegraf, InfluxDB, FastAPI, and Grafana for real-time analytics and visualization. The experimental validation successfully covered all 47 attack patterns, while maintaining stable ESP32 resource usage and low classification latency.

- **Dynamic Banner Rotation:** Introduce changing device banners and fingerprints to make the honeypot more difficult for attackers to identify.
- **Multiple SSH Sessions:** Support multiple simultaneous SSH attacker sessions.
- **NAT Traversal:** Improve deployment in networks where NAT limits direct connectivity.
- **Advanced Attack Detection:** Extend the classifier to identify more complex and previously unseen attack behaviors.
- **Scalable Deployment:** Deploy multiple ESP32 honeypot nodes for larger distributed IoT monitoring.
- **Enhanced Analytics:** Expand the real-time dashboards and analytics capabilities for deeper attack investigation.

8. REFERENCES

Weblinks:

1. https://www.espressif.com/en/products/socs/esp32
2. https://mosquitto.org/
3. https://docs.influxdata.com/influxdb/
4. https://grafana.com/docs/
5. https://docs.docker.com/

Journals:

1. C. Kolias, G. Kambourakis, A. Stavrou, and J. M. Voas, "DDoS in the IoT: Mirai and Other Botnets," *Computer*, vol. 50, no. 7, pp. 80–84, 2017, doi: 10.1109/MC.2017.201.
2. W. Zhang, B. Zhang, Y. Zhou, H. He, and Z. Ding, "An IoT Honeynet Based on Multiport Honeypots for Capturing IoT Attacks," *IEEE Internet of Things Journal*, vol. 7, no. 5, pp. 3991–3999, 2020, doi: 10.1109/JIOT.2019.2956173.
3. S. Lee, A. Abdullah, and N. Z. Jhanjhi, "A Review on Honeypot-based Botnet Detection Models for Smart Factory," *International Journal of Advanced Computer Science and Applications*, vol. 11, no. 6, 2020, doi: 10.14569/IJACSA.2020.0110654.
4. W. Wang, Y. Shang, Y. He, Y. Li, and J. Liu, "BotMark: Automated Botnet Detection with Hybrid Analysis of Flow-Based and Graph-Based Traffic Behaviors," *Information Sciences*, vol. 511, pp. 284–296, 2020, doi: 10.1016/j.ins.2019.09.024.
5. S. S. C. Silva, R. M. P. Silva, R. C. G. Pinto, and R. M. Salles, "Botnets: A Survey," *Computer Networks*, vol. 57, no. 2, pp. 378–403, 2013, doi: 10.1016/j.comnet.2012.07.021.
6. S. Khattak, N. R. Ramay, K. R. Khan, A. A. Syed, and S. A. Khayam, "A Taxonomy of Botnet Behavior, Detection, and Defense," *IEEE Communications Surveys & Tutorials*, vol. 16, no. 2, pp. 898–924, 2014, doi: 10.1109/SURV.2013.091213.00134.
7. N. Hoque, D. K. Bhattacharyya, and J. K. Kalita, "Botnet in DDoS Attacks: Trends and Challenges," *IEEE Communications Surveys & Tutorials*, vol. 17, no. 4, pp. 2242–2270, 2015.
8. R. Sundarasekar, "Hiotpot: Surveillance on IoT Devices Against Recent Threats," *Wireless Personal Communications*, vol. 103, no. 2, pp. 1179–1194, 2018.
9. W. Zhang, B. Zhang, Y. Zhou, H. He, and Z. Ding, "An IoT Honeynet Based on Multiport Honeypots for Capturing IoT Attacks," *IEEE Internet of Things Journal*, vol. 7, no. 5, pp. 3991–3999, 2020.
10. Y. Zeng *et al.*, "Using Honeypots to Model Botnet Attacks on the Internet of Medical Things," *Computers & Security*, 2022. The study used distributed honeypots to analyze temporal attack behavior and botnet control periods.

Conference:

1. Y. M. P. Pa, S. Suzuki, K. Yoshioka, T. Matsumoto, K. Kasama, and C. Rossow, “IoTPOT: Analysing the Rise of IoT Compromises,” in *Proc. 9th USENIX Workshop on Offensive Technologies (WOOT)*, 2015.
2. J. Guarnizo *et al.*, “SIPHON: Towards Scalable High-Interaction Physical Honeypots,” in *Proc. 2017 Workshop on Cyber-Physical Systems Security (CPSS)*, 2017, pp. 57–68.
3. S. Herwig, K. Harvey, G. Hughey, R. Roberts, and D. Levin, “Measurement and Analysis of Hajime, a Peer-to-peer IoT Botnet,” in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2019, doi: 10.14722/ndss.2019.23488.
4. M. A. Hakim, H. Aksu, and A. S. Uluagac *et al.*, “U-PoT: A Honeypot Framework for UPnP-Based IoT Devices,” in *Proc. International Conference on Performance Evaluation and Modeling in Wireless and Wired Networks*, 2018.
5. R. Vishwakarma, “A Honeypot with Machine Learning Based Detection Framework for Defending IoT Based Botnet DDoS Attacks,” in *Proc. 3rd International Conference on Trends in Electronics and Informatics (ICOEI)*, 2019, pp. 1019–1024.
6. “IoT Honeypot: A Review from Researcher's Perspective,” in *Proc. 2018 IEEE Conference on Application, Information and Network Security (AINS)*, 2018, doi: 10.1109/AINS.2018.8631494.

Book:

1. M. Spitzner, *Honeypots: Tracking Hackers*. Boston, MA, USA: Addison-Wesley, 2003.
2. L. Spitzner, *Honeypots: Tracking Hackers*. Indianapolis, IN, USA: Addison-Wesley, 2003.
3. E. Cole, *Honeypots: A Security Education*. Rockland, MA, USA: Syngress, 2005.